Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

Division of Electronics and Communication Engineering
2026-2027 (ODD SEM)

III IA EVALUATION REPORT

for

SYSTEM VERILOG FOR FUNCTIONAL VERIFICATION - PROJECT BASED COURSE

Title of the project: Low-Power UART Controller with Dual FIFOs and Clock-Gating using System Verilog

A report submitted by

Name of the Student	***MADAVA VENKAT J*** ***VENKATANATHAN S***
Register Number	***URK23EC1026*** ***URK23EC1015***
Subject Name	**SYSTEM VERILOG FOR FUNCTIONAL VERIFICATION**
Subject Code	***19EC2018***
Date of Report submission	***26/10/2026***

Project Rubrics for Evaluation

First Review: Project title selection - PPT should have four slides (Title page, Introduction, Circuit/Block Diagram, and Description of Project).

Second Review: PPT should have three slides (Description of Concept, implementation, outputs, results and discussion)

Rubrics for project (III IA - 40 Marks):
Content - 4 marks (based on Project)
Clarity - 3 marks (based on viva during presentation) Feasibility - 3 marks(based on project)
Presentation - 10 marks Project
Report - 10 marks
On-time submission - 5 marks (before the due date)
Online submission-GCR - 5 marks

Total marks: / 40 Marks

Signature of Faculty with date:

TABLE OF CONTENTS

+

CHAPTER	TITLE	PAGE NO.
1.	INTRODUCTION	3
2.	DESCRIPTION OF THE PROJECT	5
3.	CONCEPT INVOLVED	7
4.	TOOLS	10
5.	IMPLEMENTATION	11
6.	RESULTS WITH SIMULATION	14
7.	INFERENCES	18
8.	CONCLUSION	19
9.	REFERENCES	20

CHAPTER 1

INTRODUCTION

In the modern world, digital communication plays an important role in electronic systems, embedded devices, microcontrollers, and computer-based applications. Data must be transferred accurately and efficiently between different devices for the system to function properly. One of the most widely used methods for serial data communication is the Universal Asynchronous Receiver/Transmitter (UART). UART is commonly used because of its simple architecture, low hardware complexity, and ability to transmit and receive data without requiring a separate clock signal between communicating devices. It is widely applied in embedded systems, sensors, development boards, communication modules, and various digital electronic applications.

A UART communication system consists of two main functional blocks: a transmitter and a receiver. The transmitter converts parallel data into a serial bit stream, while the receiver converts the incoming serial data back into parallel form. Data is generally transmitted in frames containing a start bit, data bits, and a stop bit. In a typical 8-bit UART configuration, the eight data bits are transmitted one at a time, starting with the least significant bit. Since UART communication is asynchronous, the transmitter and receiver must be configured with compatible timing parameters to ensure that data is sampled and interpreted correctly.

Although UART is relatively simple, the design of an efficient UART controller involves several important considerations, including **data buffering**, **transmission timing**, **error detection**, and **power consumption**. In many digital systems, the internal clock continues switching even when a particular hardware block is not performing any useful operation. These unnecessary clock transitions can contribute to dynamic power consumption. Reducing power consumption is especially important in portable electronics, battery-operated devices, and embedded systems.

The project titled **"Low-Power UART Controller with Dual FIFOs and Clock-Gating using SystemVerilog"** focuses on designing and simulating a UART communication system that combines reliable data transfer with reduced clock activity. The proposed design consists of a **UART transmitter**, a **UART receiver**, a **transmit FIFO**, a **receive FIFO**, and a **clock-gating circuit**. The transmit FIFO temporarily stores data before transmission, while the receive FIFO stores successfully received data until it is read by the system. These buffers help manage the flow of data and allow the transmission and reception processes to operate without requiring every data operation to occur at exactly the same time.

The main low-power technique used in this project is **clock gating**. Clock gating controls the delivery of the clock signal to a selected circuit block. When the UART transmitter is idle and does not need to perform an operation, the clock-gating circuit prevents unnecessary clock transitions from reaching the transmitter. When transmission is required, the clock is enabled so that the transmitter can process and send the data. Since dynamic power consumption is related to switching activity, reducing unnecessary clock transitions can help reduce the dynamic power associated with the gated circuit.

The complete design is described using **SystemVerilog**, a hardware description language used to model and implement digital systems at the Register Transfer Level (RTL).

The overall objective of this project is to develop a functional UART controller that combines serial communication, FIFO-based data buffering, and clock gating in a single SystemVerilog design. The project also provides practical experience in modular RTL development, finite state machines, digital communication, simulation-based verification, and low-power design techniques. Through this implementation, the project demonstrates how functional correctness and clock activity reduction can be evaluated together when developing a digital communication system with low-power design considerations.

CHAPTER 2

DESCRIPTION OF THE PROJECT

The project, "Low-Power UART Controller with Dual FIFOs and Clock-Gating using SystemVerilog," focuses on designing and simulating a digital communication system that can transmit and receive serial data while reducing unnecessary clock activity. The design combines UART communication, FIFO-based data buffering, and a clock-gating mechanism into a single integrated system. The complete design is described using SystemVerilog and verified through simulation using EDA Playground.

The proposed system consists of five main functional blocks: the UART transmitter, the transmit FIFO, the UART receiver, the receive FIFO, and the clock-gating circuit. Each block performs a specific task, and all the blocks are integrated through a top-level module named *uart_dual_fifo_top*. This modular structure makes the design easier to develop, test, and maintain.

UART Transmitter

The UART transmitter is responsible for **converting parallel data into a serial data stream**. It accepts an 8-bit data value and transmits the bits sequentially through the TX output. The transmission frame consists of a start bit, eight data bits, and a stop bit. The transmitter uses a finite state machine to control the sequence of operations and generates status signals such as *tx_busy* and *tx_done* to indicate transmission activity and completion.

Transmit FIFO

The transmit FIFO stores data before it is transmitted. Instead of requiring the system to provide each byte at the exact moment it is needed, data can be written into the FIFO and transmitted when the controller is ready. The FIFO follows the First-In, First-Out principle, ensuring that data is processed in the order in which it was written. Its status signals, including full, empty, and count, provide information about buffer availability and occupancy.

UART Receiver

The UART receiver accepts the serial data stream at the RX input and **reconstructs the original 8-bit data.** It detects the start bit, samples the incoming data bits according to the configured timing, and checks the stop bit. When a valid frame is received, the receiver generates the parallel data and asserts the reception-completion signal. It also provides a framing-error signal to indicate an invalid stop bit.

Receive FIFO

The receive FIFO stores the bytes successfully received by the UART receiver. The stored data remains available until the system reads it. The FIFO provides status signals such as *rx_full*, *rx_empty*, and *rx_count*, allowing the system to monitor the buffer condition and determine when received data is available.

Clock-Gating Circuit

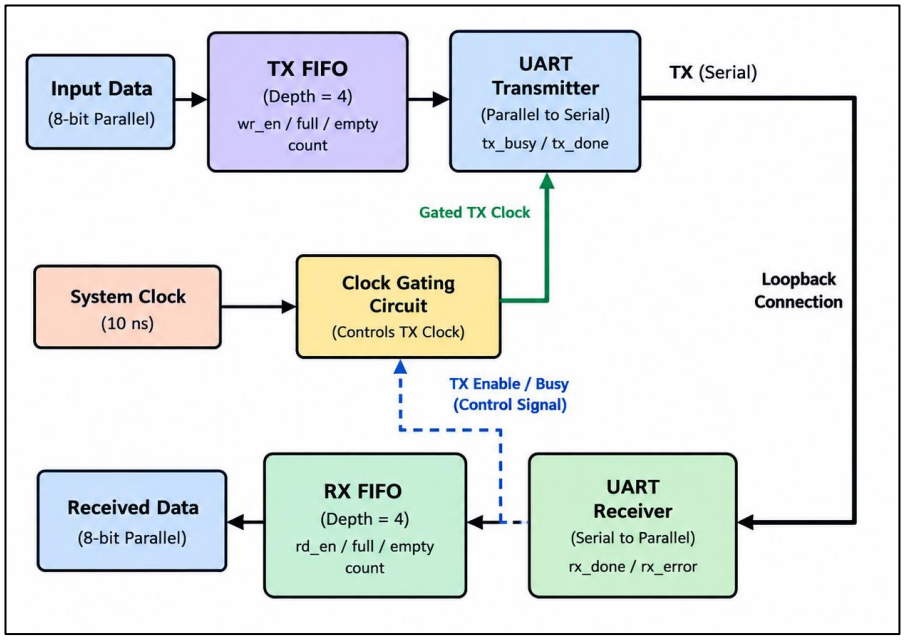
The clock-gating circuit is included to reduce unnecessary clock transitions in the UART transmitter. It controls the clock supplied to the transmitter according to the transmission-start and transmission-busy conditions. When transmission is inactive, the gated clock can be disabled; when transmission is required, the clock is enabled to allow the transmitter to operate. This reduces clock activity in the transmitter domain during periods when the transmitter does not need to perform work. The implementation used in this project is an RTL model intended for simulation and educational demonstration.

Top-Level Integration and Verification

All the functional blocks are connected through the top-level module to form a complete UART controller. The TX FIFO supplies data to the transmitter, while the receiver passes valid received bytes to the RX FIFO. The clock-gating circuit controls the transmitter's clock activity.

To verify the integrated system, a loopback connection is established by connecting the TX output directly to the RX input. Three hexadecimal data bytes, A5, 3C, and 7F, are transmitted and then checked at the receiver side. The simulation confirms that the tested bytes are received in the correct order. The testbench also counts rising edges on the source clock and gated transmitter clock to evaluate clock activity. The measured results show 274 source-clock rising edges and 243 gated-clock rising edges, representing an 11.31% reduction in rising-edge count during the measured simulation interval.

Therefore, the project demonstrates the integration of serial communication, data buffering, and clock gating in a single SystemVerilog design. It verifies the functional operation of the UART data path and evaluates the reduction in transmitter clock activity through simulation.



CHAPTER 3

CONCEPT INVOLVED

The design and implementation of a low-power UART controller involve several important digital design concepts. These include asynchronous serial communication, finite state machines, FIFO data buffering, clock gating, and Register Transfer Level (RTL) modelling using SystemVerilog.

1) Universal Asynchronous Receiver/Transmitter (UART)

UART is a serial communication technique used to transfer data between digital devices. It converts parallel data into a serial bit stream at the transmitter and reconstructs the original parallel data at the receiver. Since UART is asynchronous, a separate clock signal is not transmitted along with the data. Instead, the transmitter and receiver use compatible timing configurations to communicate reliably.

In this project, the UART uses an **8-N-1 frame format**, which consists of one start bit, eight data bits, no parity bit, and one stop bit. The start bit is logic 0, while the stop bit is logic 1. The eight data bits are transmitted one at a time, beginning with the least significant bit (LSB). The receiver detects the start bit, samples the data bits according to the configured timing, and checks the stop bit before accepting the received byte.

The simulation uses *CLKS_PER_BIT = 8* and a source clock period of 10 ns. Therefore, each UART bit occupies eight clock cycles, giving a nominal bit duration of 80 ns and a corresponding nominal baud rate of 12.5 Mbit/s.

2) Finite State Machine (FSM)

A finite state machine is a digital control structure that operates through a defined set of states. The next state depends on the current state and the relevant input conditions. FSMs are commonly used in communication controllers because they provide an organised method for controlling sequential operations.

In the UART transmitter, the states represent the **idle, start-bit, data-bit, and stop-bit** operations. When transmission begins, the controller moves from the idle state to the start state, transmits the data bits sequentially, sends the stop bit, and then returns to the idle state.

The receiver uses a separate state machine to detect the beginning of an incoming frame, sample the data bits, and validate the stop bit. Using state machines makes the transmitter and receiver behaviour easier to understand and verify.

3) First-In, First-Out (FIFO) Buffer

A FIFO is a data storage structure in which the first item written is the first item read. It is useful when data is produced and consumed at different times. In the proposed design, two FIFO buffers are used: one for transmitting data and another for receiving data.

The transmit FIFO stores bytes before they are sent through the UART transmitter. The receive FIFO stores successfully received bytes until the system reads them. This arrangement provides temporary storage and helps manage the flow of data through the communication system.

The FIFO provides status signals to monitor its condition. A *full* signal indicates that the buffer cannot accept additional data, while an *empty* signal indicates that no data is available to read. The count signal provides information about the number of stored entries. These signals help the controller and testbench monitor the availability of data and buffer capacity.

4) Clock Gating

Clock gating is a low-power design technique used to prevent unnecessary clock transitions from reaching a selected circuit block. In synchronous digital systems, clock signals drive sequential elements such as registers and state machines. Even when a block is idle, clock transitions can cause switching activity and contribute to dynamic power consumption.

In this project, a clock-gating circuit controls the clock supplied to the UART transmitter. The gate is enabled when transmission needs to start or when the transmitter is busy. When the transmitter is idle and the enable signal is inactive, the gated clock is prevented from toggling.

The clock-gating logic uses a latched enable that updates while the source clock is low. This approach helps avoid unwanted clock pulses caused by changes in the enable signal while the clock is high. The gated clock then supplies the timing required by the transmitter during operation.

5) Dynamic Power Consumption

Dynamic power is the power consumed when capacitances in a digital circuit charge and discharge as signals switch. A commonly used approximation is:

$$\text{Dynamic power consumption} = \alpha C V^2 f$$

where:

- (α) represents the switching activity factor.
- (C) represents the effective switched capacitance.
- (V) represents the supply voltage.
- (f) represents the clock frequency.

The equation shows that dynamic power depends on switching activity, capacitance, voltage, and frequency. Clock gating primarily aims to reduce unnecessary switching activity in the gated circuit. When the transmitter is idle, preventing clock transitions can reduce the clock-related switching activity of its sequential logic.

In this project, the number of rising edges on the source clock and gated transmitter clock is counted during simulation. The measured reduction indicates a decrease in clock activity over the tested interval. However, clock-edge reduction alone cannot determine the total power saved because the capacitance, supply voltage, internal switching, and clock-gating overhead are not measured directly.

6) Register Transfer Level (RTL) Design Using SystemVerilog

Register Transfer Level design describes the operation of digital hardware in terms of registers, data transfers, and control logic. SystemVerilog provides constructs for describing combinational logic, sequential logic, state machines, counters, and module connections.

In this project, each major functional block is implemented as a separate module. The transmitter, receiver, FIFOs, and clock-gating circuit are connected through a top-level module. This modular approach supports easier debugging, testing, and future modification.

Simulation allows the RTL design to be checked before any physical hardware is built. A testbench applies input data, generates the clock and reset signals, monitors the outputs, and checks the expected behaviour.

7) Loopback Testing and Functional Verification

Loopback testing is a verification method in which the transmitter output is connected directly to the receiver input. Data sent by the transmitter is therefore passed back to the receiver, allowing the complete communication path to be tested without an external UART device.

In this project, the bytes A5, 3C, and 7F are transmitted through the UART and received through the loopback connection. The testbench checks the received data, verifies the order of the bytes, monitors the FIFO status, and checks for framing errors.

The simulation also counts rising edges on the source clock and gated transmitter clock. The measured counts are 274 and 243, respectively, corresponding to an 11.31% reduction in rising-edge count during the measured interval. This verifies a reduction in clock activity for the tested simulation conditions, although it does not directly measure total circuit power.

In summary, the concepts of UART communication, FSM-based control, FIFO buffering, clock gating, dynamic power, and RTL verification form the foundation of the proposed system. Together, they enable the design to perform serial data communication while demonstrating a technique for reducing unnecessary clock activity.

CHAPTER 4

TOOLS

1) EDA Playground

EDA Playground is a free, browser-based platform for writing, compiling, and simulating hardware description languages such as Verilog and SystemVerilog. It was used as the main development environment for this project. The platform allows the design modules and testbench to be entered and executed online without requiring a complete local EDA software installation.

In this project, EDA Playground was used to develop the UART transmitter, UART receiver, TX FIFO, RX FIFO, top-level controller, and clock-gating circuit. It was also used to execute the testbench and observe the simulation results.

Project link: https://edaplayground.com/x/Ypp9\

2) SystemVerilog

SystemVerilog is a hardware description language used to model and describe digital electronic circuits. It supports the design of combinational logic, sequential logic, finite state machines, counters, and modular hardware systems.

In this project, SystemVerilog was used to implement the UART transmitter and receiver, FIFO buffers, clock-gating logic, and top-level module. It was also used to create the testbench that generates input data, controls the simulation, checks received bytes, and counts clock edges.

3) HDL Simulator – Cadence Xcelium

The HDL simulator available through EDA Playground was used to compile and execute the SystemVerilog design. Simulation helps verify the behaviour of the circuit before physical hardware implementation.

The simulator was used to test the transmission and reception of the hexadecimal data bytes A5, 3C, and 7F. It also helped verify the received data order, FIFO operation, and framing-error status. In addition, the testbench measured the source-clock and gated-clock rising edges to evaluate clock activity.

4) EPWave Waveform Viewer

EPWave is a waveform viewer that displays signal changes over simulation time. It helps the designer inspect the timing relationships between different signals and understand the internal operation of a digital circuit.

In this project, EPWave was used to observe the source clock, gated TX clock, TX output, RX input, FIFO status signals, and transmission or reception status signals. The waveforms help demonstrate how data moves through the UART controller and whether clock gating operates as intended.

CHAPTER 5

IMPLEMENTATION

Design of the UART Transmitter

The first step was to implement the UART transmitter, which converts 8-bit parallel data into a serial data stream. The transmitter uses a finite state machine with idle, start, data, and stop states to control the transmission sequence.

When a transmission begins, the transmitter sends a logic-low start bit, followed by eight data bits in least-significant-bit-first order, and then a logic-high stop bit. A timing counter controls the duration of each bit according to the *CLKS_PER_BIT* parameter. The transmitter also generates status signals, including *tx_busy* and *tx_done*, to indicate whether transmission is in progress and when it has completed.

Implementation of the TX FIFO

After developing the transmitter, a FIFO buffer was implemented to store data before transmission. The TX FIFO accepts parallel data through its write interface and makes the stored data available to the transmitter in the correct order.

The FIFO has a configured depth of four entries in the simulation. Control signals and status outputs, including *wr_en*, *full*, *empty*, and *count*, are used to manage data storage and monitor the buffer condition. This allows multiple bytes to be queued for transmission rather than requiring each byte to be supplied at the exact moment it is transmitted.

Design of the UART Receiver

The UART receiver was implemented to reconstruct parallel data from the incoming serial bit stream. It detects the start bit, uses a timing counter to determine when to sample the incoming data, and stores the eight data bits in a shift or data register.

After receiving the data bits, the receiver checks the stop bit. If the stop bit is valid, the receiver updates the received-data output and asserts *rx_done*. If the stop bit is invalid, it asserts *rx_framing_error*. This allows the design and testbench to distinguish valid frames from frames with a framing error.

Implementation of the RX FIFO

The RX FIFO was connected to the UART receiver to store successfully received bytes until they are read by the system. This separates the reception of serial data from the later operation of reading the received data.

The receive interface includes *rx_rd_en* for reading data and status signals such as *rx_full*, *rx_empty*, and *rx_count*. These signals allow the testbench to determine when received data is available and monitor the number of stored entries.

Integration of the Top-Level Module

Once the transmitter, receiver, and FIFO modules were implemented, they were connected through the top-level module named *uart_dual_fifo_top*.

The TX FIFO provides data for transmission, and the UART transmitter converts each byte into a serial frame. On the receiving side, the UART receiver converts the serial frame back into an 8-bit value and stores the received byte in the RX FIFO.

The top-level module exposes the required input, output, control, and status signals so that the complete system can be operated and verified through the testbench.

Implementation of Clock Gating

The clock-gating circuit was added to reduce unnecessary clock transitions in the UART transmitter. The gate-enable signal is based on the transmit-start and transmit-busy conditions. This allows the transmitter clock to be enabled when transmission is required and disabled when the transmitter is idle.

The RTL clock gate uses a latched enable that updates while the source clock is low. The gated clock is then connected to the transmitter's clock input. This method is intended to prevent the enable signal from changing during the high phase of the source clock and producing unwanted clock pulses.

Testbench Development

A SystemVerilog testbench was created to verify the complete design. The testbench generates a clock with a period of 10 ns and applies the required reset sequence. The UART timing parameter is configured as *CLKS_PER_BIT = 8*, and the FIFO depth is set to four entries.

Three hexadecimal data bytes, A5, 3C, and 7F, are supplied to the transmit path. The transmitter output is connected directly to the receiver input to create a loopback connection. This allows the data sent by the transmitter to be received by the same system.

The testbench waits for the expected bytes to arrive in the RX FIFO, reads them, and checks that the data values and order are correct. It also monitors the framing-error signal to verify that no framing error occurs during the valid test frames.

Clock Activity Measurement

To evaluate the clock-gating mechanism, the testbench counts the rising edges of the source clock and the gated TX clock during the same simulation interval.

The percentage reduction is calculated using the following equation:

$$\text{Edge reduction} = \frac{274 - 243}{274} \times 100\%$$

$$= \frac{31}{274} \times 100\% = \boxed{11.31\%}$$

The simulation recorded 274 rising edges on the source clock and 243 rising edges on the gated TX clock. The difference is 31 rising edges, resulting in an 11.31% reduction over the measured interval. This result demonstrates reduced clock activity in the transmitter clock domain under the tested conditions. It is an activity measurement, not a direct measurement of total power consumption.

Simulation and Verification

The integrated design was compiled and simulated using EDA Playground. The testbench results were examined, and the waveforms were viewed using EPWave to observe the source clock, gated clock, TX and RX signals, and relevant status signals.

CHAPTER 6

RESULTS WITH SIMULATION

The following parameters were used during the simulation.

Parameter	Value
Clock period	10 ns
Clock generation	always #5 clk = ~clk
UART timing parameter	CLKS_PER_BIT = 8
FIFO depth	4 entries
Number of test bytes	3
Test data	A5, 3C, 7F
Communication method	TX-to-RX loopback
Simulation platform	EDA Playground
Waveform viewer	EPWave

Table 1 : Simulation parameters

The source clock period was set to 10 ns, meaning that each clock cycle lasted 10 ns. With CLKS_PER_BIT = 8, each UART bit occupied eight clock cycles, giving a nominal bit duration of 80 ns. The TX output was connected directly to the RX input to allow the transmitted bytes to be received and checked by the testbench.

Functional Verification of UART Communication

The first part of the simulation focused on verifying the data transmission and reception process. Three hexadecimal bytes, A5, 3C, and 7F, were supplied to the transmit path. The TX FIFO stored the data before the UART transmitter converted each byte into a serial frame.

The serial output was then passed directly to the receiver through the loopback connection. The receiver reconstructed the bytes and stored them in the RX FIFO. The testbench read the received bytes and checked their values and order.

The functional verification results are shown below.

Test condition	Expected result	Observed result
Transmission of A5	Byte transmitted and received correctly	Passed
Transmission of 3C	Byte transmitted and received correctly	Passed
Transmission of 7F	Byte transmitted and received correctly	Passed
RX FIFO data order	Bytes remain in the expected order	Passed
RX FIFO read operation	Stored bytes can be retrieved	Passed
Framing-error check	No framing error for valid test frames	Passed

Table 2 : Functional verification results

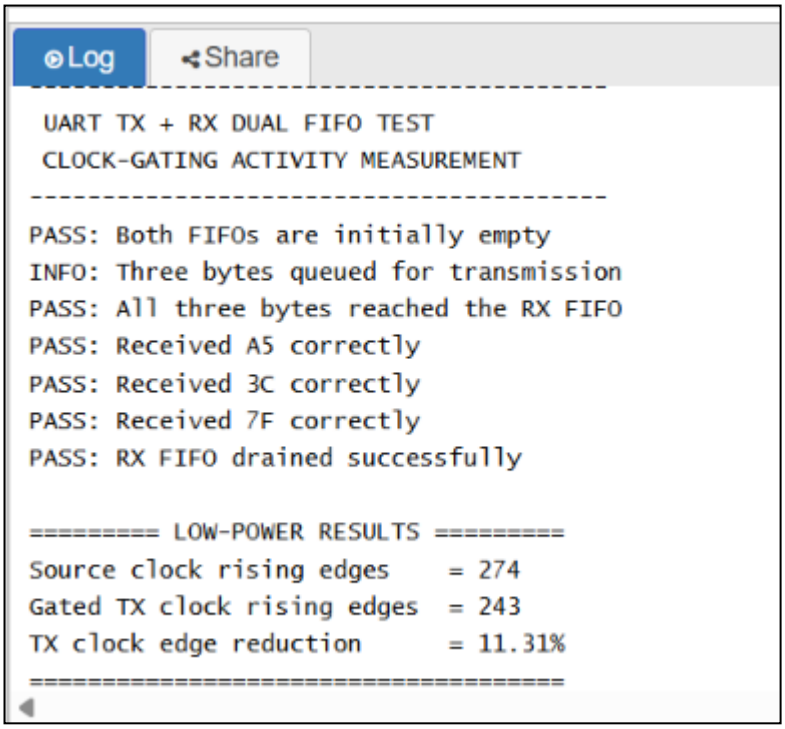


Fig 1 : Functional verification results

The simulation results confirm that the three test bytes were transmitted and received correctly under the tested conditions. The FIFO buffers operated as expected during this test, and no framing error was reported for the valid loopback frames.

Clock-Gating Simulation Results

The second part of the simulation evaluated the clock activity of the transmitter. The testbench counted the rising edges of the source clock and the gated TX clock during the same simulation interval.

The measured results are presented below.

Measurement	Result
Source clock rising edges	274
Gated TX clock rising edges	243
Difference in rising edges	31
Rising-edge reduction	11.31%

Table 3 : Clock gating simulation results

The result indicates that the gated TX clock produced 31 fewer rising edges than the source clock during the measured simulation interval. This demonstrates a reduction in clock activity within the transmitter clock domain for the tested design and conditions.

Waveform Analysis

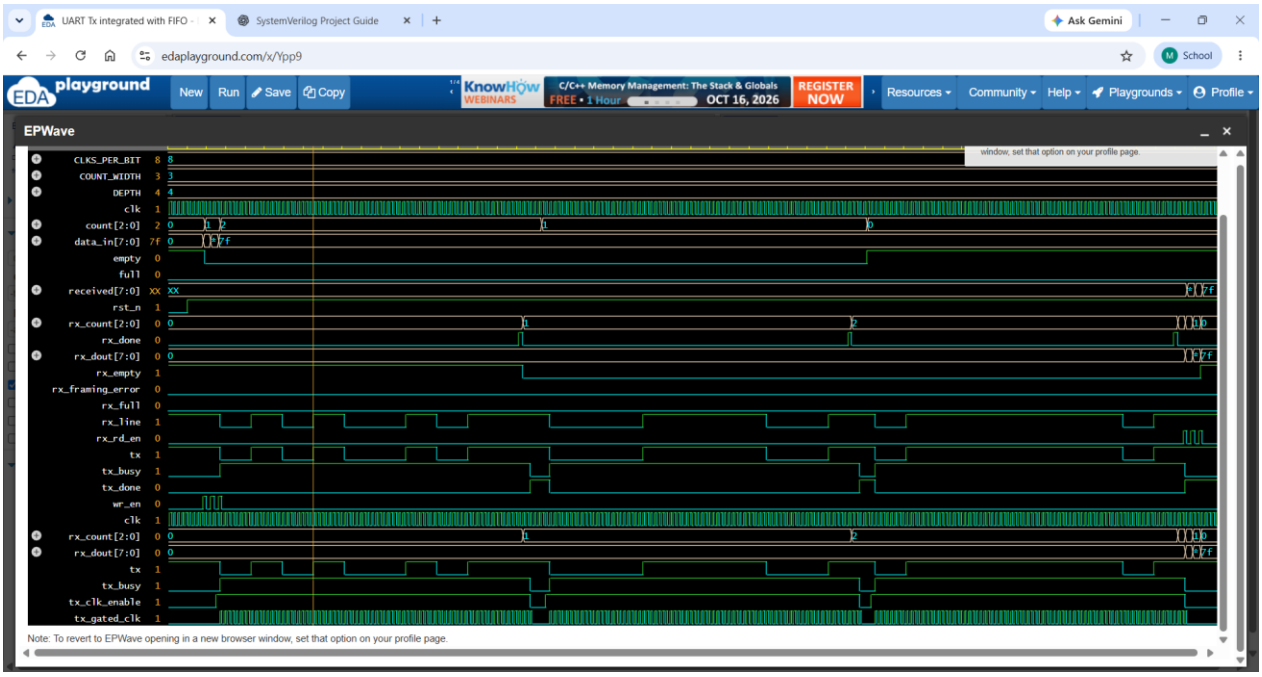


Fig 2 : Waveform results

Signal	Meaning	What your waveform shows
CLKS_PER_BIT = 8	Clock cycles per UART bit	Each serial bit is timed using 8 clock cycles.
COUNT_WIDTH = 3	Width of a counter	A 3-bit counter can represent values from 0 to 7.
DEPTH = 4	FIFO capacity	Each FIFO can hold up to 4 data words, assuming this parameter applies to its depth.
clk	Main clock	Continuously toggles to synchronize the design.
count[2:0]	Counter	Tracks a count or timing state, depending on the module.
data_in[7:0]	Input data byte	Shows data being supplied to the design, including 7F (hexadecimal).
empty	FIFO empty flag	Indicates whether the associated FIFO contains data.
full	FIFO full flag	Indicates whether the associated FIFO has reached capacity.
received[7:0]	Received data output	Shows XX in this screenshot, indicating an unknown or unresolved value at the displayed time.
rst_n	Active-low reset	A LOW level asserts reset; HIGH releases reset.
rx_count[2:0]	RX FIFO occupancy count	Increases from 0 to 1 and then 2 as data is received, before later changing.
rx_done	Receive completion pulse	Brief pulses indicate

		completed UART receive operations.
rx_dout[7:0]	RX FIFO output data	Displays 7F near the end of the waveform.
rx_empty	RX FIFO empty flag	Goes LOW when received data is available and returns HIGH when the FIFO becomes empty.
rx_framing_error	UART framing error flag	Remains LOW in the displayed waveform, so no framing error is indicated.
rx_full	RX FIFO full flag	Remains LOW; the RX FIFO is not shown as full.
rx_line	UART receive serial input	Changes between HIGH and LOW as serial data is received.
rx_rd_en	RX FIFO read enable	Indicates when the receiver FIFO is being read.
tx	UART transmit serial output	Carries the serial data frame from the transmitter.
tx_busy	Transmitter busy flag	Indicates that transmission is in progress.
tx_done	Transmit completion pulse	Brief pulses mark completion of transmission operations.
wr_en	FIFO write enable	Indicates when a write operation is requested.

Table 4 : Signal descriptions

CHAPTER 7

INFERENCES

The following inferences can be drawn from the design and simulation:

1. **Successful Data Communication:** The UART transmitter and receiver successfully transmitted and received the test bytes A5, 3C, and 7F in the correct order.
2. **Effective FIFO Buffering:** The TX and RX FIFOs stored and transferred data correctly during the tested simulation.
3. **Reduced Clock Activity:** Clock gating reduced the rising-edge count from 274 source-clock edges to 243 gated-clock edges, representing an 11.31% reduction.
4. **Potential Power Reduction:** Reduced clock activity can help lower dynamic power consumption in the transmitter domain. However, the actual power saving was not directly measured.
5. **Successful RTL Verification:** SystemVerilog simulation using EDA Playground and EPWave helped verify the design's functionality and observe its timing behaviour.

CHAPTER 8

CONCLUSION

The project **"Low-Power UART Controller with Dual FIFOs and Clock-Gating using SystemVerilog"** was successfully designed and simulated to demonstrate UART communication, data buffering, and clock activity reduction. The main objective was to develop a UART controller that can transmit and receive serial data while using clock gating to reduce unnecessary switching activity.

The design consists of a UART transmitter, a TX FIFO, a UART receiver, an RX FIFO, and a clock-gating circuit integrated into a top-level module. The transmitter converts parallel data into serial format, while the receiver converts the incoming serial data back into parallel format. The FIFOs provide temporary storage for data and help manage data transfer between the system and the UART communication blocks.

The complete design was implemented using SystemVerilog and tested on EDA Playground. Loopback testing was performed by connecting the transmitter output to the receiver input. The test bytes A5, 3C, and 7F were transmitted and received correctly in the expected order. These results confirmed the basic functionality of the UART transmitter, receiver, and FIFO-based data flow under the tested conditions.

Clock gating was incorporated to reduce clock activity in the transmitter section when continuous clocking was not required. During simulation, the source clock recorded 274 rising edges, while the gated clock recorded 243 rising edges over the same measurement interval. This corresponds to an **11.31% reduction in clock rising-edge count**. The result demonstrates that clock gating can reduce clock activity and has the potential to lower dynamic power consumption. However, the actual power consumption was not measured directly in this project.

This project provided practical experience in digital design concepts such as UART communication, finite-state-machine-based control, FIFO buffering, clock gating, and simulation-based verification. It also demonstrated how individual hardware modules can be integrated into a complete digital system and tested using a SystemVerilog testbench and waveform analysis.

In conclusion, the project achieved its main functional objectives and demonstrated the usefulness of clock gating in a UART controller. Future improvements could include testing FIFO overflow and underflow conditions, improving receiver input synchronisation, verifying reset and framing-error behaviour, and performing detailed power analysis. These enhancements would help improve the reliability and power-efficiency evaluation of the design. Overall, the project establishes a useful foundation for developing low-power serial communication controllers in digital systems.

CHAPTER 9

REFERENCES

1. Shanmugasundaram, N. (2018, March). Clock gating techniques: an overview. In *2018 conference on emerging devices and smart systems (ICEDSS)* (pp. 217-221). IEEE.
2. Wu, Q., Pedram, M., & Wu, X. (2000). Clock-gating and its application to low power design of sequential circuits. *IEEE transactions on circuits and systems I: fundamental theory and applications*, *47*(3), 415-420.
3. Yu, S., Yi, L., Chen, W., & Wen, Z. (2007, May). Implementation of a Multi-channel UART Controller Based on FIFO Technique and FPGA. In *2007 2nd IEEE Conference on Industrial Electronics and Applications* (pp. 2633-2638). IEEE.
4. Dhanasekar, S., Boopathy, A., VP, A. K., & Madhumitha, D. (2026, January). Design of Low Power UART Architecture with FIFO Buffering for High-Speed Applications. In *2026 International Conference on Signal Processing and Electronics Design (ICSPED)* (pp. 182-188). IEEE.
5. Sutherland, S., Davidmann, S., & Flake, P. (2006). *SystemVerilog for Design: A guide to using SystemVerilog for hardware design and modeling*. Boston, MA: Springer US.